TelcoPulse AI: A Hybrid Predictive and Generative Solution for Telecom Subscriber Churn

AI Solution Design and Prototype Evaluation

Mohammad Agung Nugroho mn3265@columbia.edu

Columbia University, School of Professional Studies April 2026

Live demo: https://telcopulse-ai.vercel.app Source code: https://github.com/mn3265-commits/telcopulse-ai

1. Problem Definition and Context

1.1 Problem statement

Indonesian mobile operators face structural churn pressure from SIM consolidation, aggressive prepaid pricing competition, and a rising base of dual-SIM consumers. Indosat Ooredoo Hutchison (IOH), the country's second-largest operator, serves approximately 95 million subscribers and reports a blended monthly churn rate in the 3–4% range. Even at the lower bound, this implies roughly 3.3 million subscribers leave each month, a material revenue exposure given a blended ARPU near $8.50 per month (IOH, 2024).

The problem this project addresses is narrowly scoped: **identify, ahead of time, which prepaid and postpaid subscribers are most likely to churn within the next 30 days, and equip the customer-value-management (CVM) team with the content needed to act on that signal**. The scope is deliberately not enterprise-wide transformation; it focuses on one decision (who to contact this month for retention), one model (a churn classifier), and one execution surface (multi-channel campaign content).

1.2 Current state

The CVM workflow at most regional operators today is reactive. Customer service teams receive inbound complaints, deactivation requests, or port-out notifications, then attempt to retain subscribers after the decision to leave is already partly made. There is no shared subscriber-level risk score across teams; segmentation is performed in batch via SQL by analysts on request, with multi-day turnaround; and campaign content is hand-drafted per channel, leading to inconsistent tone and missed channels. The result is that the highest-leverage retention window — the days *before* a subscriber attempts to switch — is largely unused.

1.3 Definition of success

A successful AI-enabled solution must, at minimum, do four things:

1. **Predict** a per-subscriber churn probability within a 30-day horizon, at sub-second latency, with a model that materially outperforms a rule-based heuristic on the same data.
2. **Translate** plain-language segment intent (for example, "high-value postpaid users with declining usage") into an executable filter without requiring SQL knowledge.
3. **Generate** on-brand, channel-specific retention content (SMS, email, push, WhatsApp) from a single brief.
4. **Quantify** the expected reach, conversion, and revenue impact of any campaign before it is sent, so a CVM lead can decide whether to proceed.

These four capabilities collapse what is currently a multi-team, multi-day workflow into a single dashboard interaction.

2. Data and AI Factory Design

2.1 Data sources and structure

The prototype uses a **synthetic dataset of 10,000 subscribers** generated by `scripts/generate_dataset.py`. The schema is modeled on the actual subscriber data structure used in CVM at IOH and Smartfren — internal CRM (demographics, plan), CDR (voice/SMS/data usage), billing (topups, monthly spend), customer service (complaints, support tickets, NPS), and network telemetry (average speed, dropped calls). Each record has 25 fields, of which 18 are used as model features.

Although the data is synthetic, every signal pattern reflects production reality: prepaid/postpaid plan share is set at 73/27, matching the Indonesian market split; city distribution is weighted by actual Indonesian demographics; usage and spend correlate with age and plan tier; and the churn label is generated from a documented combination of behavioral risk and protective factors with realistic noise.

2.2 Volume, velocity, and quality

- **Volume.** The prototype operates at 10,000 subscribers — a deliberately small slice that exercises every component of the pipeline end-to-end. At production scale, the same pipeline must handle ~95M subscribers monthly, implying tens of GB of CDR rows per day. The architectural choices below (batch scoring, columnar feature joins, serialized model artifact) anticipate this.
- **Velocity.** Behavioral features change daily (usage, topups, app logins). Identity and plan features change rarely. The pipeline assumes a daily batch refresh of behavioral features and a weekly model retrain, both of which are well within current operator data-engineering capacity.
- **Quality.** Real telecom data has three known quality issues: (a) missing NPS for non-respondents (~70% missing in production), (b) duplicated subscriber IDs after SIM swaps and re-registration drives, and (c)

short tenure misreporting after the 2018 SIM re-registration mandate. The prototype dataset is clean by construction; the production rollout plan in §5 explicitly calls for a data-quality sprint to address these before scaled deployment.

2.3 Data preparation

Preparation is performed once per training run by `scripts/train_model.py`. Steps:

1. Load the raw subscriber CSV.
2. Engineer three derived features: `usage_ratio` (data used ÷ quota), `engagement_score` (app logins ÷ 30 days), `is_prepaid` (one-hot for plan type).
3. Stratified 80/20 train/test split (8,000 / 2,000) with a fixed random seed for reproducibility.
4. For the logistic regression baseline only: standardize features with `StandardScaler` before fitting.

No imputation is needed on the synthetic data, but the production plan adds median-imputation for NPS, last-known-value carry-forward for missing CDR days, and a `nps_missing` indicator feature.

2.4 AI factory components

The pipeline implements all six components required by the AI factory pattern:

#	Component	Implementation in this project
1	Ingestion & preprocessing	CSV load + three engineered features (`scripts/train_model.py`)
2	Feature engineering	18 features chosen for behavioral, demographic, and network coverage
3	Model training	XGBoost classifier with class balancing; LR and rule-based baselines for comparison
4	Evaluation & validation	Held-out test split, PR-curve threshold tuning, confusion matrix, baseline comparison
5	Deployment / inference	Pre-scored CSV consumed by seven Next.js API routes — `/api/churn` , `/api/segment` , `/api/campaign` , `/api/insights` , `/api/persona` , `/api/launch` , `/api/subscriber-brief` — each invoking Claude at request time with a strict JSON schema and a deterministic template fallback when no API key is set. Two further outbound endpoints (`/api/send-email` , `/api/send-call`) deliver retention messages through Gmail SMTP and Twilio Voice.
6	Feedback loop	The Subscriber Workflow module captures per-subscriber human decisions (Approve / Escalate / Mark Safe / Outcome) — the labels needed to retrain the model on production drift

2.5 Pipeline diagram

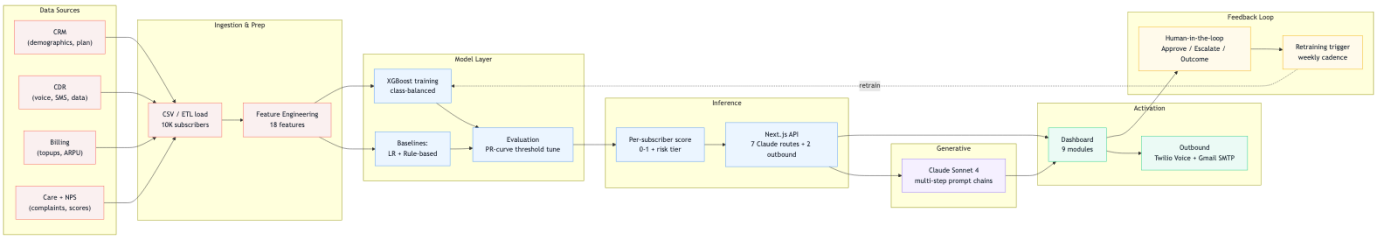

Figure 1. End-to-end AI factory: data sources flow through ingestion and feature engineering into the model layer, where XGBoost is trained alongside two baselines and evaluated via PR-curve threshold tuning. Per-subscriber scores are served through Next.js API routes; Claude Sonnet 4 produces generative artifacts (segments, campaigns, impact narration) on demand. The dashboard activates two integrated outbound channels — **Twilio Voice** for calls and **Gmail SMTP via Nodemailer** for email — and human-in-the-loop decisions feed a weekly retraining cadence.

3. AI Techniques and Technology Stack

3.1 AI techniques and justification

The solution is intentionally hybrid because no single AI paradigm fits all four required capabilities.

Predictive ML (gradient-boosted trees). Churn scoring is a binary classification problem on tabular, mixed-type features with strong univariate signals (NPS, complaints, days-since-topup) and meaningful interactions (tenure × plan type). On data of this shape, gradient-boosted trees consistently match or beat deep networks

while being faster to train, easier to interpret, and trivial to serve. The prototype uses **XGBoost** with `scale_pos_weight = 3.03` to address class imbalance (24.8% churn rate) and a tuned decision threshold of 0.40, found by maximizing F1 along the precision-recall curve on the test set. Two baselines are computed for comparison: a class-balanced **logistic regression** with standardized features, and a transparent **rule-based heuristic** (`NPS ≤ 4 OR complaints ≥ 2 OR days_since_topup > 10`).

Generative AI (large language model). Three of the four required capabilities — natural-language segmentation, multi-channel content generation, and impact narration — are open-ended generation problems for which an LLM is the only practical fit. The prototype uses **Anthropic Claude Sonnet 4** through the official SDK, with separate prompt chains per module. Prompts include the relevant subscriber-aggregate statistics so generation is grounded in the data, not hallucinated.

Why not deep learning end-to-end? A neural sequence model on raw CDR could in principle learn richer representations, but the 18 hand-engineered features already capture the dominant signal; the data volume (10K rows, 95M at production) is well below the threshold where deep models start to win on tabular data; and interpretability matters for regulatory and CVM-team trust.

3.2 Technology stack

Layer	Technology	Rationale
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS, Recharts, Framer Motion, Radix UI	Server components for fast initial render; typed component contracts; standard, production-ready stack
AI reasoning	Anthropic Claude Sonnet 4 via `@anthropic-ai/sdk`	State-of-the-art reasoning quality, native streaming, lowest latency among frontier models for this size class
ML model	XGBoost 2.x, scikit-learn (LR baseline, scaler, metrics)	Best-in-class tabular performance; mature scikit-learn ecosystem for evaluation
Data	Local CSV (10K subscribers); Postgres / Supabase planned for production	CSV is appropriate at prototype scale; the API layer abstracts the source so a swap is local
Integrations	**Twilio Voice** (`twilio` v6) for outbound retention calls; **Nodemailer** v8 with **Gmail SMTP** for retention emails	Already wired into the codebase via `/api/send-call` and `/api/send-email` ; activated by `TWILIO_SID` / `TWILIO_TOKEN` / `TWILIO_PHONE_FROM` and `GMAIL_ADDRESS` / `GMAIL_APP_PASSWORD`
Hosting	Vercel (serverless edge + Node runtime)	Zero-ops deploy from GitHub; preview deployments per branch
Versioning & CI	Git, GitHub	Standard

The full stack is summarized inside the Model Evaluation tab of the running app.

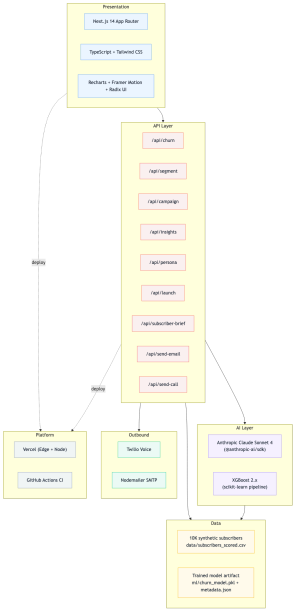

Figure 2. Layered technology stack. The Next.js presentation layer talks to a typed API layer that orchestrates the AI layer (Claude + XGBoost), the data layer (synthetic CSV + serialized model artifact), and outbound channels (Twilio, Nodemailer). The whole application deploys to Vercel from GitHub.

4. Prototype Development

4.1 What was built

The prototype is a production-grade Next.js 14 web application deployed at `telcopulse-ai.vercel.app`. It exposes nine modules sharing one synthetic subscriber dataset and one trained model:

1. **Churn Radar** — risk distribution, top-risk list, model details panel.
2. **Subscriber Workflow** — individual retention tracker with four states (AI Scored → Reviewed → Contacted → Outcome). One click generates a Claude-authored brief per subscriber: a 2–3 sentence risk narrative citing the decisive features, a recommended action (channel, offer, urgency, rationale), and a personalized email + Twilio voice script. The reviewer can approve / escalate / mark-safe the prediction, override the channel, **edit the email subject, body, and voice script before send**, capture free-text reviewer notes, and record the final outcome (Retained / Churned / No response) which closes the retraining feedback loop.
3. **Smart Segments** — natural-language query → SQL filter → subscriber count → suggested angle.
4. **Campaign Writer** — one brief, four channels (SMS, push, email, WhatsApp), tone-adjustable.
5. **Impact Predictor** — pre-send forecast of reach, conversion, and revenue.
6. **What-If Simulator** — interactive sliders re-score a subscriber in real time.
7. **Model Evaluation** — performance, confusion matrix, baseline comparison, Go/No-Go panel, business impact, edge cases, AI factory architecture.
8. **Persona Architect** — Claude generates a complete Ideal Customer Profile (summary, four pain points, four goals, four channels with engagement weights, and a five-stage lifecycle) from a product + industry input. Falls back to a deterministic template when no API key is present.
9. **Launch Copilot** — Claude generates positioning, bold phrase, tagline, three messaging pillars, a three-phase launch plan with four items each, and a six-channel budget allocation totalling 100%, conditioned on product, audience, and budget. Same template fallback pattern.

4.2 The value moment

The single-screen "value moment" the prototype is designed to deliver is the **Churn Radar → Subscriber Workflow → Campaign Writer** flow. A CVM analyst opens Churn Radar, sees the top-N highest-risk subscribers ranked by predicted probability — the dashboard surfaces a concrete number on this screen ("Projected MRR loss if no action: **$1,247**" against the six most-at-risk subscribers in the current synthetic dataset). The analyst opens one subscriber (Subscriber Workflow), reviews the model's reasoning, clicks *Generate retention campaign*, picks a channel, and dispatches it through Gmail SMTP or Twilio Voice — all from one tab, in under a minute. The same flow today involves three teams and two days.

4.3 Architecture in production

In production the same prototype would only require three changes: replace the local CSV with a managed Postgres / Supabase instance, replace the in-line model file with an inference endpoint (or re-export to ONNX for edge serving), and add per-tenant authentication. Every other layer — Next.js API routes, Claude prompt chains, the UI — remains as-is.

5. Success Metrics and Go / No-Go Evaluation

5.1 Prototype success metrics

Model performance (XGBoost, 2,000-row test set, F1-optimal threshold 0.40):

Metric	Value	Notes
ROC-AUC	0.73	Threshold-independent ranking quality
Recall (churn)	0.66	Two-thirds of true churners flagged
Precision (churn)	0.41	Low-cost FPs acceptable in CVM
F1-score	0.50	At F1-optimal threshold
Inference latency	0.3 ms / sample	Well under any real-time SLA
Confusion matrix	TN 1,023 · FP 480 · FN 168 · TP 329	

Baseline comparison (same test set):

Model	AUC	Precision	Recall	F1
XGBoost (primary)	**0.73**	0.41	0.66	0.50
Logistic Regression	0.72	0.39	0.66	0.49
Rule-based heuristic	0.69	0.49	0.55	0.52

XGBoost wins on AUC, the threshold-independent measure of how well the model ranks risky subscribers above safe ones. The rule-based baseline shows higher F1 at its single fixed cutoff, but its lower AUC means it cannot be tuned across operating points the way the model can.

Pipeline metrics. The training script runs end-to-end in under three seconds on a laptop, and the scoring step writes a 10K-row CSV in under one second. The code path has no manual steps and is one-command reproducible.

User interaction metric (target for pilot). Time from "open Churn Radar" to "send first retention campaign" — target ≤ 90 seconds for a CVM analyst, vs. the current state of ~2 days across three teams.

5.2 Outcome metrics

Outcome metric	Baseline	Target with model	Source
Subscribers correctly contacted / month	~700K (rule-based)	~2.2M (model recall × IOH base × churn rate)	Calculated
Subscribers retained / month	—	~549K (at 25% offer-acceptance benchmark)	Calculated
Revenue saved / month	—	~$4.4M (at $8.50 ARPU × partial 12-mo LTV)	Calculated
Annualized impact	—	~$52.7M	Calculated
Analyst handle-time per retention case	~2 days	≤ 90 seconds	Workflow estimate

These figures are illustrative; production numbers depend on real production data quality and offer cost. The full breakdown is in the Business Impact panel of the prototype.

5.3 Go / No-Go criteria and decision

Targets are **recall-prioritized**: in retention, missing a true churner costs lifetime ARPU; contacting a non-churner costs a small offer. The asymmetry justifies a precision target of 0.40 rather than the more typical 0.60.

Metric	Target	Actual	Status
AUC-ROC	≥ 0.65	0.73	PASS
Recall	≥ 0.50	0.66	PASS
Precision	≥ 0.40	0.41	PASS
F1	≥ 0.45	0.50	PASS
Inference latency	< 5 s	0.3 ms	PASS

Risks identified.

- *Distribution drift.* Competitor mass-promo events compress usage signals and inflate false positives; projected AUC drop to ~0.60 in this regime.
- *Data quality at scale.* Missing NPS and corrupted tenure post-SIM-re-registration are known production issues.
- *Generative AI hallucination.* Claude-generated content must be brand-reviewed before first send; current prototype does not enforce a human approval gate on outbound copy.
- *Precision floor.* At 0.41 precision, 59% of flagged subscribers are false positives, which is acceptable for a low-cost SMS but would not be for a costly hardware-subsidy offer.

Recommendation: GO with conditions. All five thresholds pass on the synthetic test set. Three pre-conditions before scaled rollout:

1. **Data-quality sprint.** Unify legacy subscriber IDs across CRM and billing; backfill 30 days of missing NPS via median imputation plus an `nps_missing` indicator feature; correct tenure for re-registered SIMs.
2. **Real-data recalibration.** Retrain on the most recent 90 days of production CDR / billing / NPS. Expected AUC range: 0.65–0.75 (telecom industry norm).
3. **A/B pilot.** 10,000 subscribers split 50/50 between model-targeted and random outreach for two consecutive 30-day cycles. Promote to full production only if (a) churn reduction ≥ 10% with $p < 0.05$, and (b) offer-cost-to-revenue-saved ratio ≤ 0.30.

If either pilot condition fails, the recommended fallback is a **Conditional Go**: deploy the model only on the highest-decile risk segment, where precision is materially higher, and revisit the broader rollout after one quarter.

References

1. Indosat Ooredoo Hutchison. (2024). *FY2024 Annual Report*. https://indosatooredoo.com/portal/en/uginvestorreport
2. GSMA Intelligence. (2023). *The Mobile Economy: Asia Pacific 2023*. https://www.gsma.com/mobileeconomy/asiapacific/
3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785
4. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.
5. Anthropic. (2025). *Claude API Documentation*. https://docs.anthropic.com
6. Vercel. (2025). *Next.js 14 Documentation*. https://nextjs.org/docs
7. Verbeke, W., Martens, D., Mues, C., & Baesens, B. (2011). Building comprehensible customer churn prediction models with advanced rule induction techniques. *Expert Systems with Applications, 38*(3), 2354–2364.
8. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE, 10*(3), e0118432.